

HydraBot Stack

Multi-agent AI infrastructure powered by Claude Code CLI

Version **1.0** Platform **Ubuntu 22.04 / Debian 12** Requires **Claude Max subscription**

0 Overview

HydraBot is a deployable AI agent stack. Each bot is a Telegram interface powered by Claude Code CLI, connected to a shared memory system (AI-IQ) and a multi-agent mesh (Circus). Ephemeral sub-workers handle background tasks in parallel.

Telegram API | ▼ PM2 Bots (grammy + Claude Code CLI) | ── circus-bridge.mjs ──> Circus API :6200 (FastAPI, systemd) | | | ▼ | AI-IQ memory (SQLite + vectors) | ── bot-circus/dispatch.mjs ──> Ephemeral Claude workers (up to 10 parallel)

LAYER	COMPONENT	PURPOSE
0	Claude Code CLI	AI inference engine — all bots call this
1	AI-IQ	Per-agent long-term memory
2	Circus	Agent mesh: registry, rooms, task queue
3	Bot-Circus	Ephemeral worker pool for sub-tasks
4	circus-bridge.mjs	Node.js ↔ Circus integration module
5	Bots	Telegram interface + business logic

1 Prerequisites

REQUIREMENT	MINIMUM	NOTES
OS	Ubuntu 22.04 / Debian 12	macOS with Homebrew also works
RAM	1 GB	2 GB recommended if running multiple bots
Disk	5 GB free	10 GB recommended
CPU	1 core	2 cores recommended
Node.js	22.x	Installer handles this automatically
Python	3.10+	Installer handles this automatically
Claude subscription	Claude Max	Required. claude.ai/upgrade — used for auth
Telegram bot token	One per bot	Free — create via @BotFather



Claude Max is required

The Claude Code CLI authenticates against your claude.ai account. A Max subscription is needed for the API access the bots rely on. The installer will pause and prompt you to log in if not already authenticated.

2 Clone the Repository

```
# Clone to /opt/hydrabot (recommended) or any directory
git clone <repo-url> /opt/hydrabot
cd /opt/hydrabot
```



You can install to any path. The installer respects the `HYDRABOT_DIR` environment variable. Examples in this guide use `/opt/hydrabot`.

3 Run the Installer

```
# Run as root (or with sudo)
sudo HYDRABOT_DIR=/opt/hydrabot bash /opt/hydrabot/deploy/install.sh
```

The installer performs these steps automatically:

- 1 Installs system packages: `curl git python3 python3-pip sqlite3 build-essential`
- 2 Installs Node.js 22 via NodeSource (skips if already installed)

- 3 Installs PM2 globally and configures systemd startup
- 4 Installs Claude Code CLI via npm (skips if already installed)
- 5 Installs Python packages: `ai-iq circus-agent`
- 6 Bootstraps the Circus data directory and generates a keypair
- 7 Installs bot-circus dependencies and initialises performer workspaces
- 8 Installs and starts the Circus API as a systemd service



Auth pause

If Claude Code CLI is not already authenticated, the installer will stop and print:

```
Claude Code CLI installed but needs authentication.  
Run 'claude' interactively to authenticate, then re-run this script.
```

Open a new terminal, run `claude`, complete the browser login with your Claude Max account, then re-run the installer command.

VERIFY INSTALLATION

```
# Circus API should be healthy  
curl http://localhost:6200/health  
# → {"status": "ok"}  
  
# Claude CLI should respond  
claude -p "say hello"  
  
# PM2 should be running  
pm2 list
```

4 Configure Your First Bot

GET A TELEGRAM BOT TOKEN

- 1 Open Telegram and search for `@BotFather`
- 2 Send `/newbot` and follow the prompts
- 3 Copy the token — it looks like `7123456789:AAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`

GET YOUR TELEGRAM USER ID

- 1 Search for `@userinfobot` in Telegram

- 2 Send it any message — it replies with your numeric ID

CREATE AND CONFIGURE THE BOT

```
# Copy the template
cp -r /opt/hydrabot/bots/template /opt/hydrabot/bots/mybot

# Copy and edit the env file
cp /opt/hydrabot/bots/template/.env.example /opt/hydrabot/bots/mybot/.env
nano /opt/hydrabot/bots/mybot/.env
```

REQUIRED .ENV VALUES

VARIABLE	REQUIRED	DESCRIPTION
TELEGRAM_BOT_TOKEN	REQUIRED	Token from @BotFather
ALLOWED_USER_ID	REQUIRED	Your numeric Telegram user ID (from @userinfobot)
BOT_NAME	OPTIONAL	Display name for the bot — default: MyBot
CLAUDE_CLI_PATH	OPTIONAL	Path to <code>claude</code> binary. Default: <code>claude</code> (uses PATH). Get with <code>which claude</code>
CLAUDE_WORKING_DIR	OPTIONAL	Working directory for Claude. Default: current directory
CLAUDE_TIMEOUT	OPTIONAL	Response timeout in ms. Default: 120000 (2 min)

START THE BOT

```
# Install dependencies
npm install --prefix /opt/hydrabot/bots/mybot

# Start with PM2
pm2 start /opt/hydrabot/bots/mybot/bot.mjs --name mybot

# Save so it survives reboot
pm2 save

# Watch logs
pm2 logs mybot
```



Bot is running

Open Telegram, find your bot, and send it a message. It should respond via Claude within a few seconds.

5 Autonomous Worker (Optional)

The worker polls a job queue and executes long-running tasks via Claude. Useful for background operations triggered from Telegram (`/task` style commands).

```
# Configure
cp /opt/hydrabot/performers/worker/.env.example \
  /opt/hydrabot/performers/worker/.env
nano /opt/hydrabot/performers/worker/.env

# Install deps
npm install --prefix /opt/hydrabot/performers/worker

# Start
pm2 start /opt/hydrabot/performers/worker/worker.mjs --name worker
pm2 save
```

VARIABLE	REQUIRED	DESCRIPTION
TELEGRAM_BOT_TOKEN	OPTIONAL	If set, worker sends job completion messages via Telegram
TELEGRAM_CHAT_ID	OPTIONAL	Your chat ID — where job results are sent
CLAUDE_CLI_PATH	OPTIONAL	Path to <code>claude</code> binary
CLAUDE_WORKING_DIR	OPTIONAL	Directory Claude operates in during jobs

6 Running Multiple Bots

Each bot is an independent directory. Repeat the configure step for each one:

```
# Second bot
cp -r /opt/hydrabot/bots/template /opt/hydrabot/bots/designbot
cp /opt/hydrabot/bots/template/.env.example /opt/hydrabot/bots/designbot/.env
# Edit .env with different TELEGRAM_BOT_TOKEN
npm install --prefix /opt/hydrabot/bots/designbot
pm2 start /opt/hydrabot/bots/designbot/bot.mjs --name designbot
pm2 save
```



All bots share one Claude Code CLI process pool and one Circus mesh. Each bot gets its own Telegram token, allowed user, and performer workspace.

7 Troubleshooting

SYMPTOM	CAUSE	FIX
Bot doesn't respond	Claude not authenticated	Run <code>claude -p "hello"</code> manually. If it fails, run <code>claude login</code>
<code>claude: command not found</code>	CLI not in PATH	Run <code>which claude</code> — if empty, run <code>npm install -g @anthropic-ai/claude-code</code> . Then set <code>CLAUDE_CLI_PATH</code> in <code>.env</code>
Circus health check fails	Service not running	<code>systemctl status circus-api</code> — if failed, check <code>journalctl -u circus-api -n 50</code>
Bot starts then crashes	Missing env var	<code>pm2 logs mybot</code> — look for "is required" errors. Check <code>.env</code> has <code>TELEGRAM_BOT_TOKEN</code> and <code>ALLOWED_USER_ID</code>
Auth works locally but not on VPS	SSH session has no browser	Run <code>claude login</code> — it prints a URL. Open that URL in any browser to complete OAuth
Circus not connecting	Wrong URL	Default is <code>http://127.0.0.1:6200</code> . Set <code>CIRCUS_URL</code> in <code>.env</code> if Circus runs elsewhere

USEFUL COMMANDS

```
pm2 list           # all running processes
pm2 logs mybot     # live bot logs
pm2 restart mybot  # restart a bot
pm2 stop mybot     # stop a bot
systemctl status circus-api # Circus API status
curl http://localhost:6200/health # quick Circus health check
claude -p "hello"  # test Claude CLI directly
```

8 Directory Reference

```
hydrabot/
├─ deploy/
│   ├─ install.sh           # Full VPS installer – run this first
│   ├─ circus-api.service  # systemd unit for Circus API
│   └─ ecosystem.config.cjs # PM2 ecosystem config
├─ bot-circus/
│   ├─ dispatch.mjs        # Ephemeral worker pool (shared by all bots)
│   └─ setup-performers.mjs # Initialise performer workspaces
├─ performers/
│   ├─ template/           # Blank performer workspace
│   ├─ webbs/              # Frontend designer bot (full example)
│   └─ worker/             # Autonomous job worker
├─ bots/
│   └─ template/           # ← start here when creating a bot
├─ circus-bridge.mjs       # Circus integration (imported by bots)
└─ docs/
    ├─ architecture.md
    ├─ configuration.md
    └─ ai-iq.md
```